🚚 DeliverIQ

AI root-cause agent for a Flipkart-style e-commerce network: capacity looks adequate, yet deliveries are late. Where does the delay really come from? (Synthetic illustrative data.)

Network utilization
68%

Late orders (now)
20.5%

Late orders (with levers)
20.5%

Complaints /1000
31

Levers: improve the whole network

Close staffing gap: **0**%

Localize inventory (cut transfers): **0**%

Optimize dispatch & routes: **0**%

Late-order estimates scale with the remaining excess delay hours; they are model estimates, not guarantees.

Fulfilment nodes – ranked by late orders (click a row)

Inventory allocation Pick & pack Dispatch wait Last mile

Node	Util.	Staff fill	Transfers	Excess delay (h)	Late %
Delhi NCR FC	67%	98%	10%	9.6	31%
Mumbai (Bhiwandi) FC	65%	101%	31%	8.3	26%
Hyderabad FC	84%	77%	12%	7.2	24%

Lucknow FC	78%	73%	22%	9.8	28%
Chennai FC	75%	97%	14%	6.3	14%
Bengaluru FC	70%	104%	8%	2.3	9%
Kolkata FC	57%	105%	9%	1.8	7%
Pune FC	45%	133%	6%	1.1	5%

Delhi NCR FC – Last mile

Inventory allocation	**+1.8 h**
Pick & pack	**+0.3 h**
Dispatch wait	**+3.5 h**
Last mile	**+4.0 h**

Root cause: Route and last-mile inefficiency: long routes and too few riders.

Utilization 67% (90,000/day capacity), staff fill 98%, 47 complaints per 1000 orders.

Recommended actions

- Dispatch: add wave dispatch and linehaul vehicles to cut wait to 2 h
- Last mile: re-optimize routes (7.8 km per delivery vs 4.0 benchmark) and add riders

Agent

Root causes | Capacity paradox | Action plan

Ask the agent. It reasons over the live numbers above.

What drives late orders? (correlation across nodes)

Inventory allocation	**r = 0.59**
Pick & pack	**r = 0.56**
Dispatch wait	**r = 0.68**
Last mile	**r = 0.75**

Algorithm used by the agent

1. **Ingest** per fulfilment node: orders/day, capacity, pickers staffed vs needed, share of

1. **Ingest** per fulfilment node: orders/day, capacity, pickers staffed vs needed, share of orders needing inter-warehouse transfer, pick-pack hours, dispatch wait, last-mile hours, km per delivery, late-order % and complaints.
2. **Decompose** end-to-end delay into four stages and compute excess over target: `allocation = 0.18 h × transfer %`, `pick–pack = hours − 4`, `dispatch = wait − 2`, `last mile = hours − 8`.
3. **Capacity paradox test**: network utilization = orders ÷ capacity, plus the spread across nodes. Adequate average with high spread means imbalance, not shortage.
4. **Attribute** the root cause per node as the stage with the largest excess; confirm network-wide with Pearson correlation of each stage's excess against the late-order rate.
5. **Staffing check**: `fill = staffed ÷ needed`; below 90% explains slow pick-pack.
6. **Rank** nodes by late orders = `orders × late %`.
7. **Recommend** per cause: staffing → shift or hire pickers; inventory → re-slot and allocate from nearest stock; dispatch/last mile → wave planning, route re-optimization, more riders; spare-capacity nodes absorb overflow.
8. **Simulate** levers: new late % = old × (remaining excess ÷ old excess), re-aggregated across the network.